Assignment 5

Purpose

For this assignment, you will utilize the skills acquired in Modules 6 to query a separate database by writing a series of complex queries. You will be responsible for testing your code before submission.

Submission

You will submit a total of **1** sql files to the Assignment 5 Submission link to CodeGrade. Files should be named appropriately and be in .sql format. Each file must use the postgres standards taught in the course.

- ***File 1***: You must submit a SQL document called <LastName>_Assignment5_Part1 to the Assignment 5 CodeGrade link. This document must include Queries 1- 7 found in part 1 of this assignment.

Instructions

Part 1: Queries Using the Banking Database

- In this part, you will populate the banking database you created in Assignment 4 with the *Assignment 5 Data*. Once the database is populated, you will be tasked with writing a series of complex queries. These queries will involve complex techniques such as JOINs, SET operators, and Subqueries.

Important Note

For Part 1 of this assignment, you will need to test your queries using a successfully completed Assignment 4 Banking Database. If you're having issues inserting the data or your queries do not run, you may need to go back and fix your Banking database or Assignment 4 DDL.

PART 1 Banking Database

For this section, you will need to use the SQL data file located in the Assignment 5 folder on Brightspace. You will use this data to populate the database you have created using your Banking DDL in Assignment 4.

Important: *Note that when a query specifies the use of a JOIN, subquery, SET operator, or a combination of these techniques, your query* ***must*** *incorporate the required method(s) at least once. This is the minimum requirement to ensure you are practicing the module concepts. However, you are not limited to using only that technique, unless otherwise specified. Feel free to employ the required technique(s) multiple times or combine it with other techniques to achieve the desired outcome. For example, if a question mandates the*

use of a join, you may opt to use multiple joins, a join with a subquery, or any other valid approach as long as the minimum requirement is met.

Queries

Query 1:

Write a query to find all customers who have at least one loan and one deposit account. Include the **cust_ID**, **account_number**, and **loan_number** in your results and organize the results by **cust_id.** Note: Some customers may appear multiple times due to having multiple loans or deposit accounts and that is ok. *Your solution must include a JOIN.*

Query 2:

Write a query that identifies all customers who have a deposit account in the same city in which they live. The results should include the **cust_id**, **customer_city**, **branch_name, branch_city**, and **account_number**. Note: The city of a deposit account is the city where its branch is located. *Your solution must use a JOIN.*

Query 3:

Write a query that returns the **cust_ID** and **customer_name** of customers who hold at least one loan with the bank, but do not have any deposit accounts. Organize your output by **cust_id.** *Your solution must use a subquery and a SET operator.*

Query 4:

Write a query to retrieve a list of **branch_names** for every branch that has at least one customer living in 'Harrison' that has a deposit account with them. Branch names should not be duplicated. Organize your results alphabetically**.** *Your solution must include a JOIN.*

Query 5:

Write a query to obtain the **cust_ID** and **customer_name** for all customers residing at the same **customer_street** address and in the same **customer_city** as customer '12345'. Exclude customer '12345' in the results and organize the results by **cust_id**. Avoid hardcoding the address for customer '12345' as their information might change. *Your solution must include a subquery and SET operator.*

Query 6:

Write a query to return each **cust_ID** and **customer_name** who has a deposit account at *every* branch located in Brooklyn. Do not hardcode the number of branches or the branch names- if the names or numbers of branches change, the query should still be valid. *Your solution must include a subquery.*

***Hint**: If you're finding this question challenging, consider utilizing aggregate functions. Think about how you could check a comparison between the number of branches per customer and the total number of branches. Remember not to hardcode these numbers.*

Query 7:

Write a query to retrieve the **loan_number, customer_name,** and **branch_name** of customers who have a loan at the Yonkahs Bankahs branch and whose loan amount exceeds the average loan amount for that branch. *Your solution must include a JOIN and a subquery*.

***Hint**: Remember, money data types do not perform well with aggregation. You will have to CAST twice here- on both sides of the comparison.*